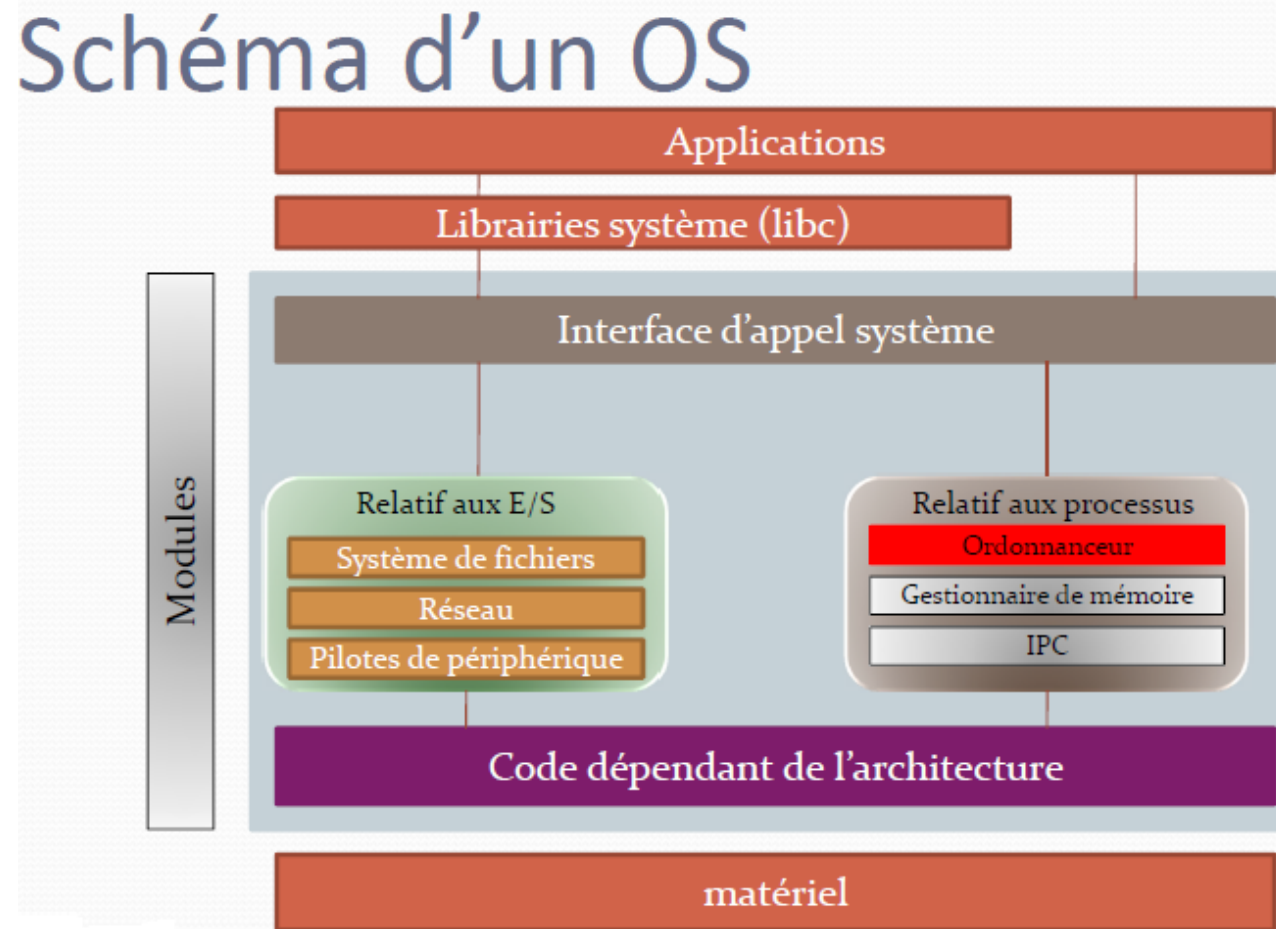

Notes

- ❑ introduction
- ❑ Concepts de Base/multiprogrammation et caractéristiques de tâches
- ❑ Critères d'Ordonnancement temps réel
- ❑ Ordonnancement TR des tâches périodiques et indépendantes
- ❑ Algorithmes d'Ordonnancement
- ❑ Ordonnancement TR des tâches apériodiques et indépendantes
- ❑ TD

Introduction /Ordonnancement des processus



Introduction

- ❏ Ordonnancement des processus
 - Système à temps partagé/temps réel, plusieurs processus peuvent être présents en mémoire en attente d'exécution.
 - Le SE (ord) doit gérer l'allocation du processeur aux différents processus à exécuter (ordonnancement).
 - L'ordonnanceur est activé par un événement :
 - ✓ interruption du temporisateur,
 - ✓ interruption d'un périphérique,
 - ✓ appel système ou signal.

L'Objectif d'ordonnancement

Objectifs de l'ordonnanceur

- Assurer que chaque processus en attente d'exécution reçoive sa part de temps processeur.
- Minimiser le temps de réponse.
- Utiliser le processeur à 100%.
- Utilisation équilibrée des ressources.
- Prendre en compte des priorités.
- Être prédictibles.

 Il est impossible de créer un algorithme qui optimise tous les critères de façon simultanée.

Métriques clés

- ❑ **Temps de réponse de l'ordonnaceur:**

temps pris par l'ordonnanceur pour effectuer le changement de contexte pour une tâche prête, ce temps inclut le temps d'attente de la tâche dans la file des tâches prêtes.

- ❑ **Turnaround time :**

Le temps pris par le processus pour compléter son exécution (capacité) .

- ❑ **Débit:** nombre de tâches traitées en un temps donné.

- ❑ **Équité:** quels sont les facteurs déterminant quant au choix de la tâche à exécuter.

- ❑ **Famine:** un ordonnanceur doit assurer que ce problème ne se produise pas...

- ❑ **Préemptivité et non préemptivité**

Problématique de l'ordonnancement temps réel

- ❑ En fonctionnement normal, respecter les contraintes temporelles spécifiées par toutes les tâches
- ❑ En fonctionnement anormal, limiter les effets des débordements temporels et assurer le respect des contraintes temporelles des tâches les plus critiques.

Dans la suite, les temps nécessaires à la mise en œuvre de l'algorithme d'ordonnancement ainsi que celui de changement de contexte soient négligeables ce qui implique une complexité faible et une implémentation efficace

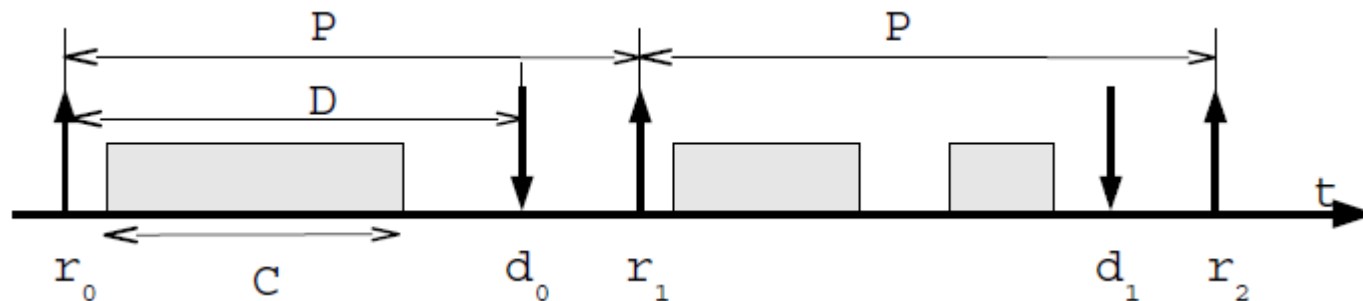
Définitions

- ❑ Tâches dépendantes ou indépendantes
 - Les tâches indépendantes ne partagent que le processeur
 - Les tâches dépendantes partagent d'autres ressources ou sont reliées par des contraintes de précédence.

Dans la suite, la mise en œuvre des algorithmes d'ordonnancement pour les tâches indépendantes

Concepts/ Modèle de tâches temps réel (1/4)

- r : date de réveil
- C : durée d'exécution maximale (capacité)
- D : délai critique
 - délai maximum acceptable pour son exécution
- P : période (si tâche périodique)
- $d = r + D$: échéance (si tâche à contraintes strictes)
- tâche périodique : $r_k = r_0 + k * P$
 - si $D = P$, tâche à échéance sur requête



Multiprogrammation, Modèle de tâches (2/4)

- $T(r, C, D, P)$ avec $0 \leq C \leq D \leq P$ // *tâche temps réel*
- Un processus est déterministe si $C = \text{cte}$,
- Un processus est préemptible s'il peut être interrompu :
 - ✓ une section d'instructions non préemptibles est appelée section critique.
- Les contraintes temporelles sont :
 - ✓ faibles si $D \gg C$,
 - ✓ fortes si $D = C + \varepsilon$ (et $\varepsilon \ll C$).

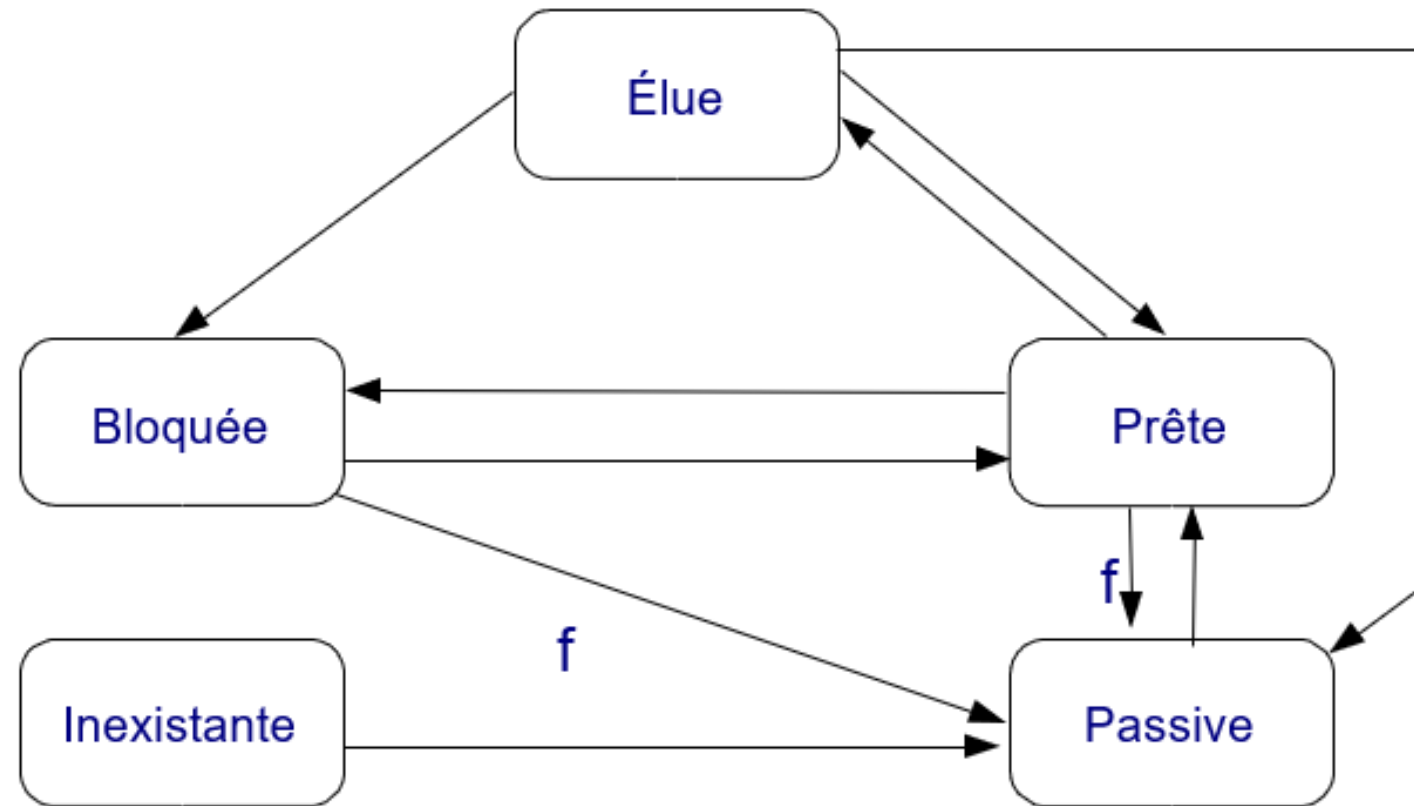
Multiprogrammation, Modèle de tâches (3/4)

- ❑ paramètres dynamiques
 - s : date du début de l'exécution
 - e : date de la fin de l'exécution
 - $D(t) = d - t$: délai critique résiduel à la date t ($0 \leq D(t) \leq D$)
 - $C(t)$: durée d'exécution résiduelle à la date t ($0 \leq C(t) \leq C$)
 - $L = D - C$: laxité nominale de la tâche
 - ✓ retard maximum pour son début d'exécution
 - $L(t) = D(t) - C(t)$: laxité nominale résiduelle
 - ✓ retard maximum pour reprendre l'exécution
 - $TR = e - r$: temps de réponse de la tâche
 - $CH(t) = C(t)/D(t)$: charge résiduelle ($0 \leq CH(t) \leq C/P$)

Multiprogrammation, Modèle de tâches (4/4)

- tâche éluë : un processeur est alloué à la tâche
 - ✓ elle peut s'exécuter
 - ✓ $C(t)$ et $D(t)$ diminuent, $L(t)$ est constant
- tâche bloquée : la tâche est en attente d'une ressource ou d'une synchronisation
 - ✓ $L(t)$ et $D(t)$ diminuent
- tâche prête : la tâche est en attente du processeur
 - ✓ $L(t)$ et $D(t)$ diminuent
- tâche passive : la tâche n'a pas de requête en cours
- tâche inexistante : la tâche n'est pas créée

État d'une tâche/mécanisme d'ordonnancement



f : abandon de la requête pour cause de faute temporelle

Définitions

□ configuration : ensemble de n tâches mises en jeu par l'application

➤ facteur d'utilisation du processeur

$$U = \sum_{i=1}^n \frac{C_i}{P_i}$$

➤ facteur de charge

$$CH = \sum_{i=1}^n \frac{C_i}{D_i}$$

➤ Condition nécessaire

$$U = \sum_{i=1}^n \frac{C_i}{P_i} \leq 1$$

□ intervalle d'étude : intervalle de temps minimum pour prouver l'ordonnançabilité d'une configuration

➤ Le PPCM des périodes dans le cas d'une configuration de tâches périodiques

Taxonomie des algorithmes d'ordonnancement

□ En ligne/hors ligne

- Hors-ligne: réalisé avant le lancement du système
- En-ligne: réalisé pendant le fonctionnement

□ préemptif ou non préemptif

- une tâche peut perdre le processeur (au profit d'une tâche plus prioritaire) ou non
- algorithme préemptif $\Leftrightarrow$ toutes les tâches préemptibles

□ Optimal/Non optimal

- Optimal: fourni un ordonnancement si un ordonnancement existe
- Non optimal (best effort): essaie de se rapprocher de l'optimal

Propriétés des algorithmes

- séquence valide
 - ✓ toutes les tâches respectent leurs contraintes
- configuration ordonnançable
 - ✓ il existe au moins un algorithme produisant une séquence valide
- algorithme optimal
 - ✓ capable de produire une séquence valide pour toute configuration de tâches ordonnançable
- test d'ordonnançabilité
 - ✓ pour vérifier qu'une configuration de tâches périodiques soumise à un algorithme d'ordonnancement peut être ordonnancée selon une séquence valide
- test d'acceptabilité
 - ✓ pour vérifier qu'une nouvelle requête peut être ajoutée à la séquence déjà formée

algorithmes d'ordonnancement Temps Réel

Algorithmes d'ordonnancement des taches périodiques

- Rate Monotonic (RM)
- Deadline Monotonic (DM)
- Earliest Deadline First (EDF)
- Least Laxity First (LLF)

Ordonnancement des tâches indépendantes

& périodiques

Ordonnancement des tâches indépendantes

❑ tâches indépendantes :

- pas de partage de ressources
- pas de contraintes de précédence

❑ cas des algorithmes en ligne

- dynamique
- sur la base d'une priorité définie
 - ✓ soit de manière empirique
 - ✓ soit à partir d'un paramètre temporel de la tâche
- priorité constante ou variable avec le temps
- à priorité identique, on évite la préemption
- test d'acceptabilité hors ligne si tous les paramètres sont connus
- sinon test de garantie au réveil des tâches

Quelques algorithmes d'ordonnancement classiques

❑ FIFO (à priorités dynamiques)

- Les tâches sont traitées dans l'ordre de leur demande d'activation.
- Toutes les tâches ont la même importance pour FIFO.
- FIFO minimise le temps de réponse de l'ensemble des tâches.
- FIFO n'est pas optimal.

❑ Round Robin (à priorités dynamiques)

- Le processeur est attribué à tour de rôle aux tâches actives.
- RR est équitable.
- RR n'est pas optimal pour des tâches temps réel.

❑ FIFO et RR ne garantissent pas les contraintes temporelles dans le cas général

Ordonnancement des tâches indépendantes

Algorithme en ligne à priorité constante

Rate Monotonic Analysis (RMA) (1/5)

- ❑ Algorithme en ligne à priorité constante
- Tâches à échéance sur requête :
La tâche de plus petite période est la plus prioritaire
- Test d'acceptabilité (condition suffisante)

$$\sum_{i=1}^n \frac{C_i}{p_i} \leq n * (2^{1/n} - 1)$$

quand n (nombre de tâches) est très grand : $n(2^{1/n} - 1) \sim \ln 2 = 0,69$

- Dans la pratique, on peut rencontrer des ordonnancements valides qui vont jusqu'à 88%
- l'utilisation de la période limite l'utilisation aux seules tâches à échéance sur requête
- On peut montrer que RMA est un algorithme « optimal » :
 - ✓ une configuration qui ne peut pas être ordonnancée par RMA ne pourra pas être ordonnée par un autre algorithme à priorité fixe

Ordonnancement RMA (2/5)

➤ Caractéristiques du RMA

Si la(les) condition(s) d'ordonnançabilité sont satisfaites:

- On calcule la priorité de chaque tâche comme suit:

$$Pr_i = 1/P_{ei}$$

- Puis l'ordonnanceur sélectionne le processus avec la plus haute priorité.

Ordonnancement RM (3/5)

Exemple 1

	T1	T2	T3
r0	0	0	0
C	3	2	2
Pe	20	5	10

$$\sum \frac{C_i}{P_i} = 0.75 \leq n(2^{1/n} - 1) = 0.77$$

$Pr(T2) > Pr(T3) > Pr(T1)$

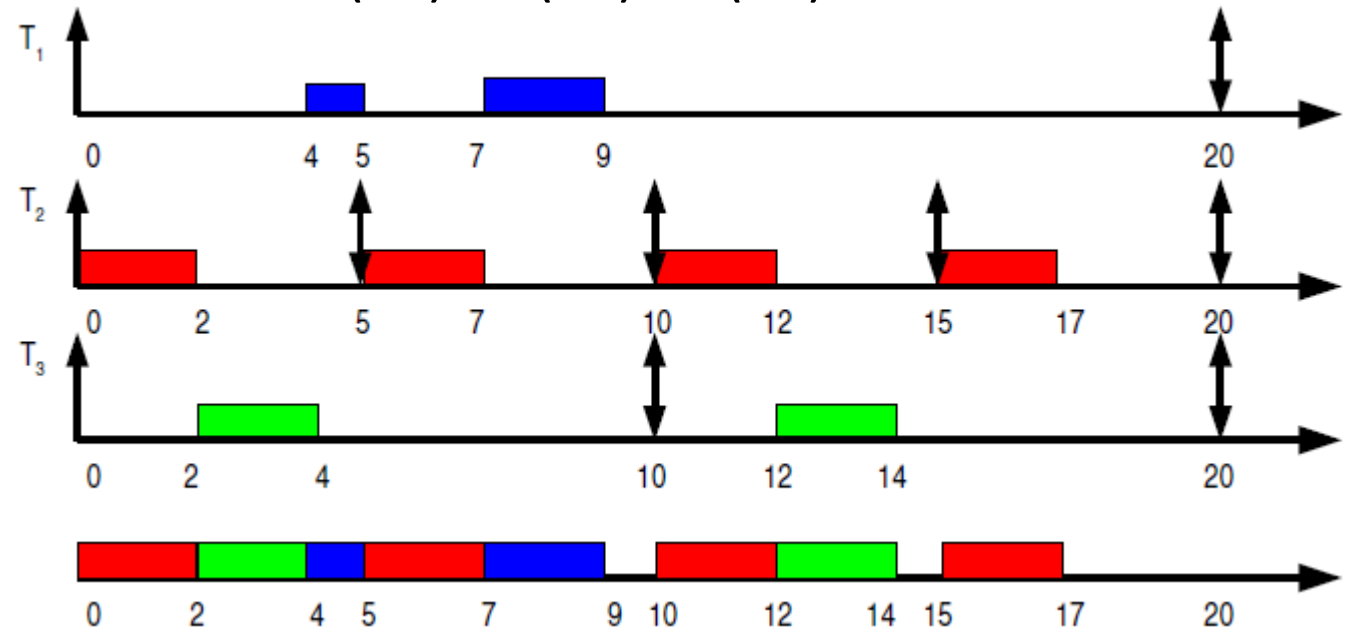


Diagramme de Gantt

Ordonnancement RMA (4/5)

Exemple 2

Tâche	C_i	Pe_i	Pr_i (Calculée par RM)
A	5	10	$1/10 = 0.1$
B	8	20	$1/20 = 0.05$
C	2	30	$1/30 = 0.033$

$$U = \sum_{i=1}^n \frac{C_i}{P_{ei}} = 0.966 > 3 * (2^{1/3} - 1) = 0.779$$

- ✓ Condition d'ordonnançabilité **non** satisfaite
- ✓ Ordonnançable néanmoins (à vérifier)

Ordonnancement RMA (5/5)

Exercice

Les trois tâches sont déclenchées à $t=0$

1. Vérifier l'ordonnançabilité
2. Donner le diagramme de Gantt

Tâche	C_i	Pe_i	Pr_i (Calculée par <i>RM</i>)
A	3	10	$1/10 = 0.1$
B	4	15	$1/15 = 0.06$
C	2	20	$1/20 = 0.05$

Ordonnancement Deadline Monotonic (DM) (1/3)

❏ algorithme en ligne à priorité constante

- basé sur le délai critique :
 - ✓ la tâche de plus petit délai critique est la plus prioritaire
- équivalent à RMA dans le cas des tâches à échéance sur requête, meilleur dans les autres cas
- Condition d'ordonnançabilité de DM
 - ✓ La condition (suffisante) d'ordonnançabilité devient:
$$\sum_{i=1}^n \frac{C_i}{D_i} < n * (2^{1/n} - 1)$$
 - ✓ La tâche doit se terminer avant l'échéance D_i
 - ✓ La condition générale nécessaire d'ordonnançabilité est toujours valable

Ordonnancement DM (2/3)

Algorithme DM

Si la(les) condition(s) d'ordonnançabilité sont satisfaites:

- ✓ On affecte la priorité aux tâches selon l'ordre inverse de leurs échéances.

$$Pr_i = 1/D_i$$

- ✓ Puis l'ordonnanceur sélectionne le processus p avec la plus haute priorité.

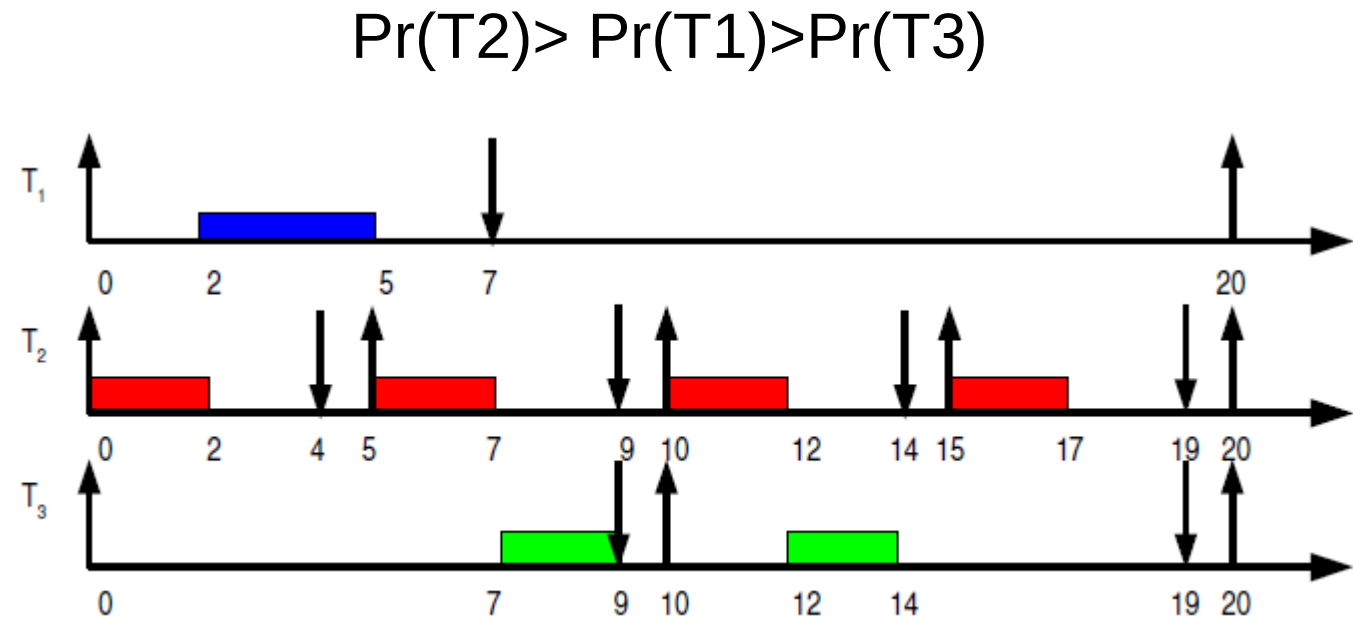
Ordonnancement DM (3/3)

Exemple

	T1	T2	T3
r0	0	0	0
C	3	2	2
D	7	4	9
Pe	20	5	10

$$\sum_{i=1}^n \frac{C_i}{D_i} = 1.14 > n * (21/n - 1) = 0.77$$

- ✓ Condition d'ordonnançabilité **non** satisfaite
- ✓ Ordonnançable néanmoins



Ordonnancement Earliest Deadline First (EDF) (1/3)

- ❑ Algorithme en ligne à priorité variable
 - Échéance la plus proche d'abord
 - Supporte des tâches apériodiques
 - base sur l'échéance
 - ✓ A chaque instant (à chaque réveil de tâche), la priorité maximale est donnée à la tâche dont l'échéance est la plus proche
 - test d'acceptabilité
 - ✓ condition nécessaire
$$U = \sum_{i=1}^n \frac{C_i}{P_i} \leq 1$$
 - ✓ condition suffisante
$$\sum_{i=1}^n \frac{C_i}{D_i} \leq 1$$

ordonnancement Earliest Deadline First (EDF) (1/4)

□ algorithme a priorité variable ou dynamique

➤ condition suffisante

$$\sum_{i=1}^n \frac{C_i}{D_i} \leq 1$$

➤ La condition devient "**nécessaire et suffisante**" si: $\forall i, D_i = P_{ei}$

➤ Le processeur peut être occupé à 100%

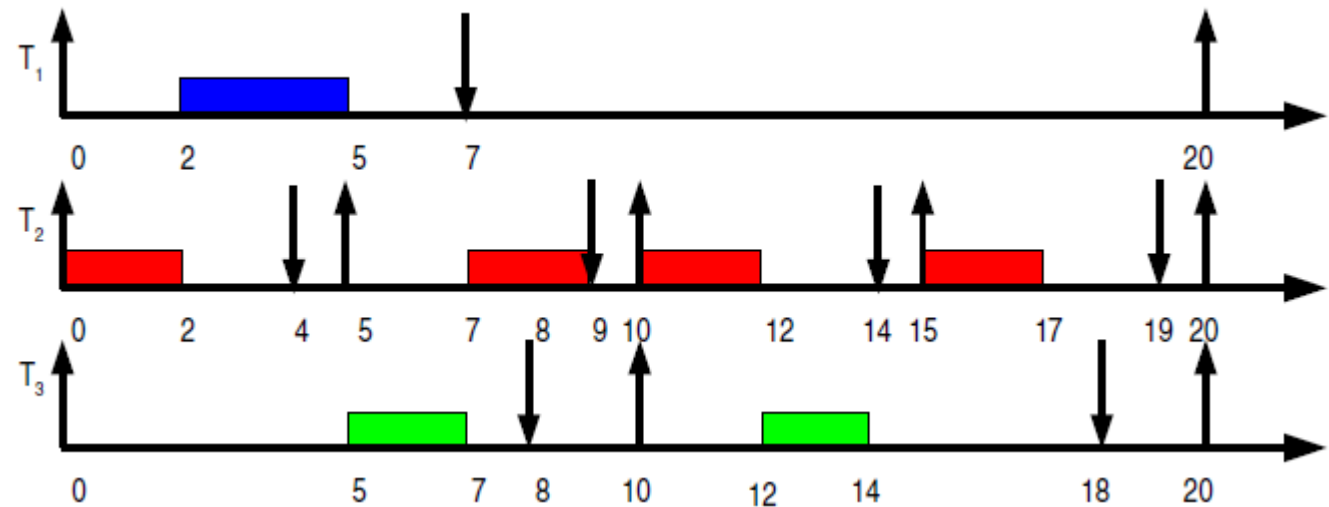
Ordonnancement EDF (2/4)

Exemple

$$\sum_{i=1}^n \frac{C_i}{P_i} = 0.75 < 1$$

	T1	T2	T3
r0	0	0	0
C	3	2	2
D	7	4	8
Pe	20	5	10

$$\sum_{i=1}^n \frac{C_i}{D_i} = 1.18 > 1$$



- ✓ Condition d'ordonnançabilité **non** satisfaite
- ✓ Ordonnançable néanmoins

Ordonnancement EDF (3/4)

□ Avantages :

- Ordonnancement optimal en ce qui concerne l'utilisation du processeur (utilisation à 100%)
- Peut traiter des tâches apériodiques

□ Désavantages :

- Implémentation difficile, il faut réajuster les priorités au fil du temps en testant les différentes échéances.
- Le temps de réponse ne se calcule pas facilement.

Ordonnancement EDF (4/4)

□ exercice

soit une configuration avec 2 taches a échéance sur requête :

T1 : ($r_0 = 0$, $C_1 = 2$, $P_1 = 5$)

T2 : ($r_0 = 0$, $C_2 = 4$, $P_2 = 7$)

1. intervalle d'étude ?
2. la configuration est-elle ordonnançable avec RM ? Justifiez votre réponse?
3. même question avec EDF
4. tracer les chronogrammes en indiquant les éventuelles fautes temporelles

ordonnancement Least Laxity First (LLF) (1/3)

- ❑ algorithme à priorité variable
 - aussi appelé Least Slack Time (LST)
 - basé sur la laxité résiduelle
- ❑ la priorité maximale est donnée à la tâche qui a la plus petite laxité résiduelle

$$L(t) = D(t) - C(t)$$

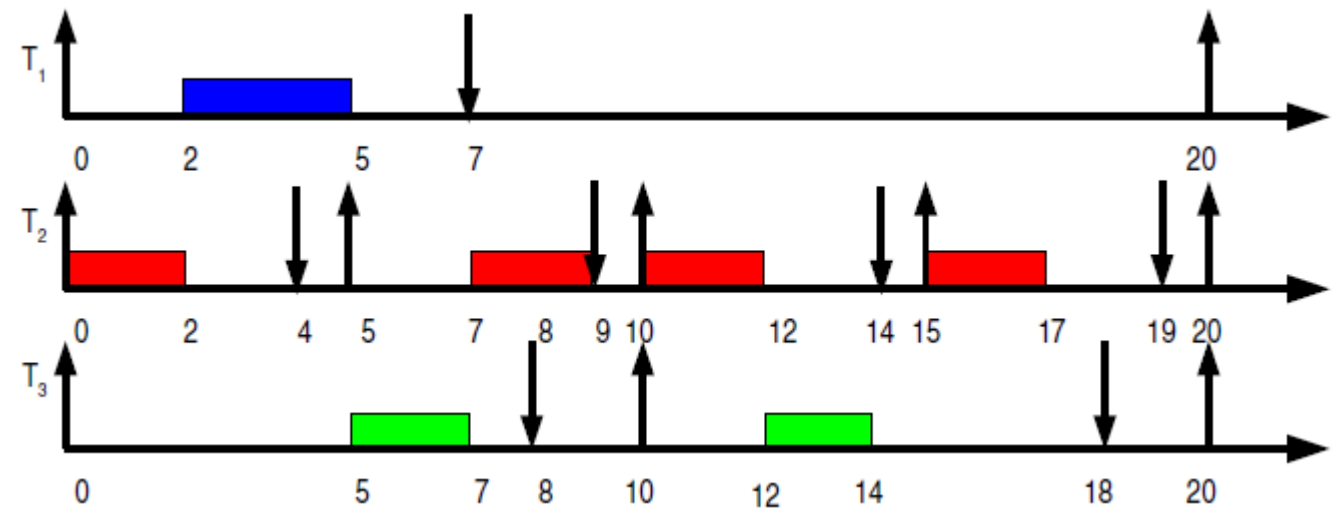
- équivalent à EDF si on ne calcule la laxité qu'au réveil des
- ❑ tâches
 - optimum à trouver entre la granularité du calcul et le
- ❑ nombre de changements de contexte provoqués

RTOS/Ordonnancement Temps Réel

ordonnancement LLF (2/3)

Exemple

	T1	T2	T3
r0	0	0	0
C	3	2	2
D	7	4	8
Pe	20	5	10



Prise en compte des tâches apériodiques

Taches apériodiques

Prise en compte des tâches apériodiques

- Tâches apériodiques: on connaît pas l'instant d'arrivée de la requête de réveil
 - ◆ contraintes temporelles strictes ou relatives
- Tâche sporadiques: se caractérisent par un délai minimum entre deux activations
- buts à atteindre :
 - ◆ si contraintes relatives : minimiser le temps de réponse
 - ◆ si contraintes strictes : maximiser le nombre de tâches acceptées en respectant leurs contraintes
- 2 grandes catégories de traitement
 - ◆ Traitement en arrière-plan
 - ◆ Traitement par serveurs

Ajout de tâches apériodiques

□ Principes

- Traiter les tâches apériodiques comme les tâches périodiques lorsque l'ordonnancement le permet
- Le test d'ordonnancement sert à tester l'admission dès lors que les tâches apériodiques viennent avec des caractéristiques plus ou moins précises

□ Hypothèses

- Le test d'ordonnancement (pas l'algorithme d'ordonnancement) doit être de faible complexité pour que l'admission s'effectue dynamiquement

Ordonnancement des Tâches apériodiques à contraintes relatives

❑ traitement en arrière-plan

- Le plus simple, mais le moins performant
- Tâches apériodiques ordonnancées quand le processeur est oisif
- Si plusieurs tâches attendent, elles sont traitées en mode FIFO

Tâches apériodiques à contraintes relatives

◆ exemple de traitement en arrière-plan

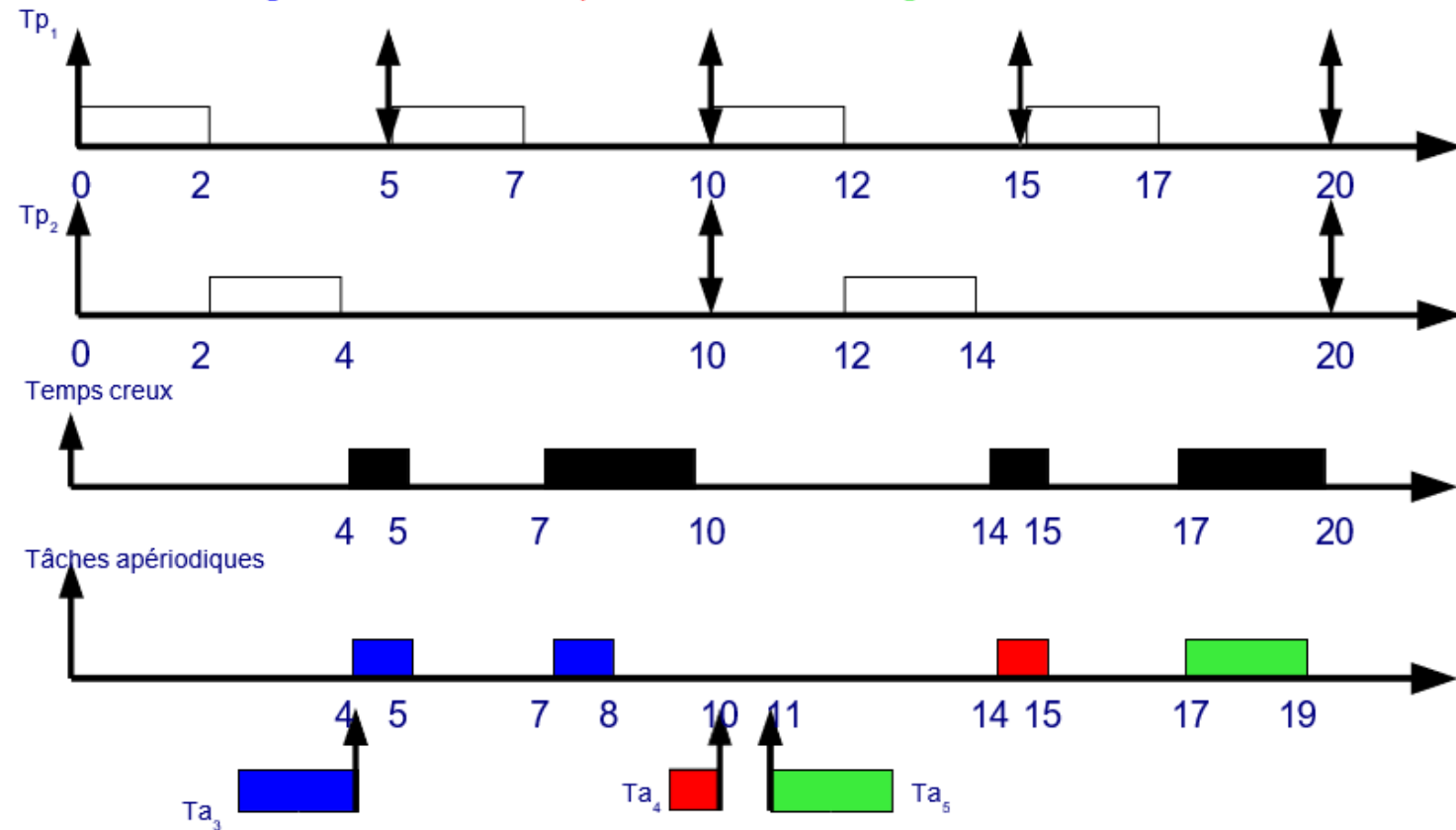
Tp_1 ($r_0 = 0$, $C = 2$, $P = 5$),

Tp_2 ($r_0 = 0$, $C = 2$, $P = 10$)

Ta_3 ($r = 4$, $C = 2$),

Ta_4 ($r = 10$, $C = 1$),

Ta_5 ($r = 11$, $C = 2$)



Tâches apériodiques à contraintes relatives

- ❑ Traitement par serveur
 - un serveur est une tâche périodique créée spécialement pour prendre en compte les tâches apériodiques
 - serveur caractérisé par
 - ✓ sa période
 - ✓ son temps d'exécution : *capacité du serveur*
 - ✓ serveur généralement ordonnancé suivant le même algorithme que les autres tâches périodiques
 - ✓ une fois actif, le serveur sert les tâches apériodiques dans la limite de sa capacité. L'ordre de traitement des tâches ne dépend pas de l'algorithme général

Tâches apériodiques à contraintes relatives

❑ Serveurs par scrutation (*polling*)

- ◆ à chaque activation, traitement des tâches en suspens jusqu'à épuisement de la capacité ou jusqu'à ce qu'il n'y ait plus de tâches en attente
- ◆ si aucune tâche n'est en attente (à l'activation ou parce que la dernière tâche a été traitée) , le serveur se suspend immédiatement et perd sa capacité qui peut être réutilisée par les tâches périodiques

Tâches apériodiques à contraintes relatives

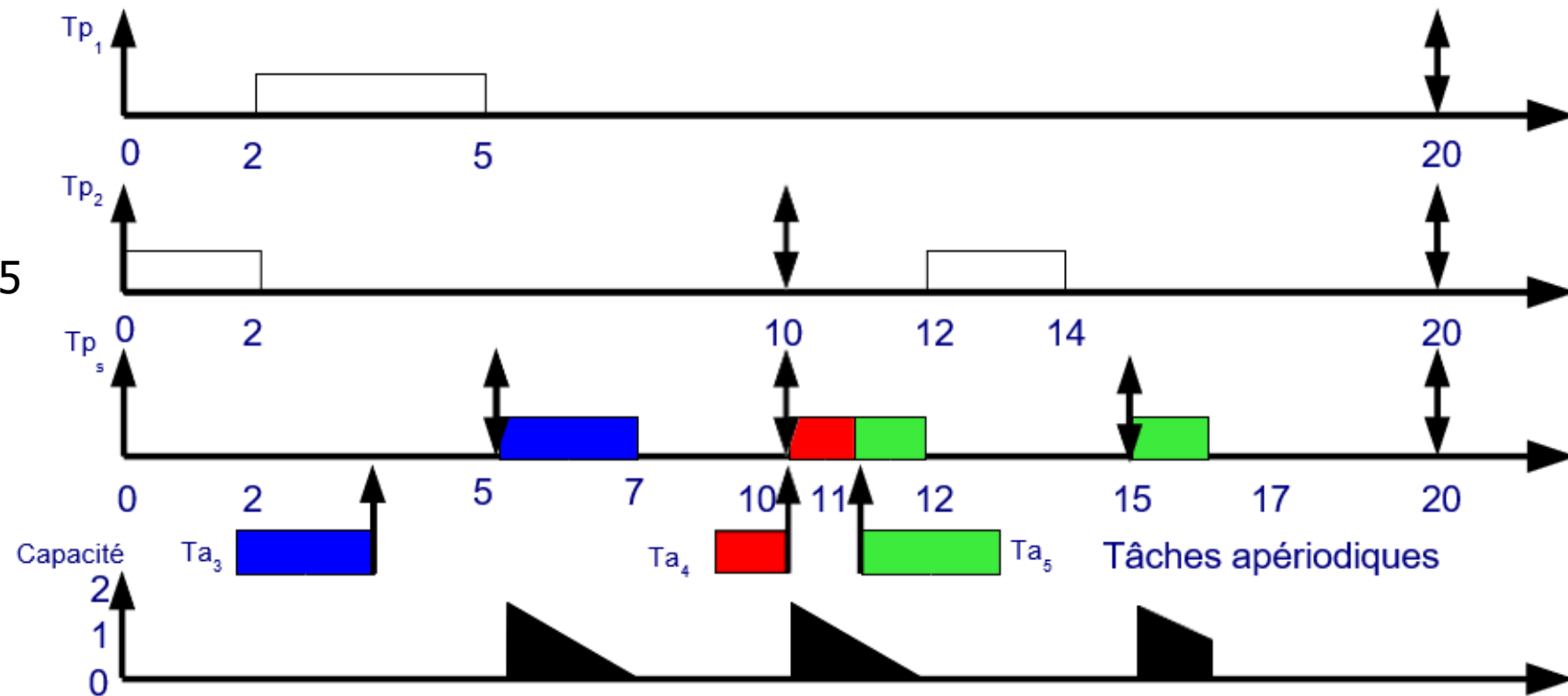
□ exemple de serveur par scrutation (ordonnancement RMA)

➤ 2 tâches périodiques :

- ◆ $Tp_1 (r_0 = 0, C = 3, P = 20)$
- ◆ $Tp_2 (r_0 = 0, C = 2, P = 10)$
- ◆ serveur : $Tp_s (r_0 = 0, C = 2, P = 5)$

➤ tâches apériodiques :

- ◆ $Ta_3 (r = 4, C = 2)$,
- ◆ $Ta_4 (r = 10, C = 1)$,
- ◆ $Ta_5 (r = 11, C = 2)$



Tâches apériodiques à contraintes relatives

❑ limitations du serveur par scrutation

- ◆ perte de la capacité si aucune tâche apériodique en attente
- ◆ si occurrence d'une tâche apériodique alors que le serveur est suspendu, il faut attendre la requête suivante

❑ serveur ajournable

- ◆ la capacité n'est pas rendue aux tâches périodiques
- ◆ permet de traiter une tâche apériodique qui survient juste après la requête de réveil du serveur
- ◆ mais viole la règle essentielle selon laquelle la tâche de plus haute priorité doit s'exécuter en premier
- ◆ peut entraîner des fautes temporelles
- ◆ critère d'acceptabilité:

$$U_p \leq \ln\left(\frac{U_s + 2}{2U_s + 1}\right), U_s = \frac{C_s}{P_s}, U_p = \sum_{i \in TP} \frac{C_i}{P_i}$$

Tâches apériodiques à contraintes relatives

❑ Serveur sporadique

Principe

- ◆ Les tâches apériodiques sont traitées séquentiellement par un serveur de forte priorité
- ◆ Le serveur dispose d'une capacité et d'une période
- ◆ Le temps consommé à traiter une tâche apériodique est débité sur la capacité
- ◆ Le montant exact de temps consommé est crédité après un délai d'une période
- ◆ Le serveur devient actif lorsqu'une tâche apériodique est à traiter et que sa capacité n'est pas épuisée

Tâches apériodiques à contraintes relatives

▣ Serveur sporadique

➤ 2 tâches périodiques :

Tp_1 ($r_0 = 0, C = 3, P = 20$),

Tp_2 ($r_0 = 0, C = 2, P = 10$)

serveur :

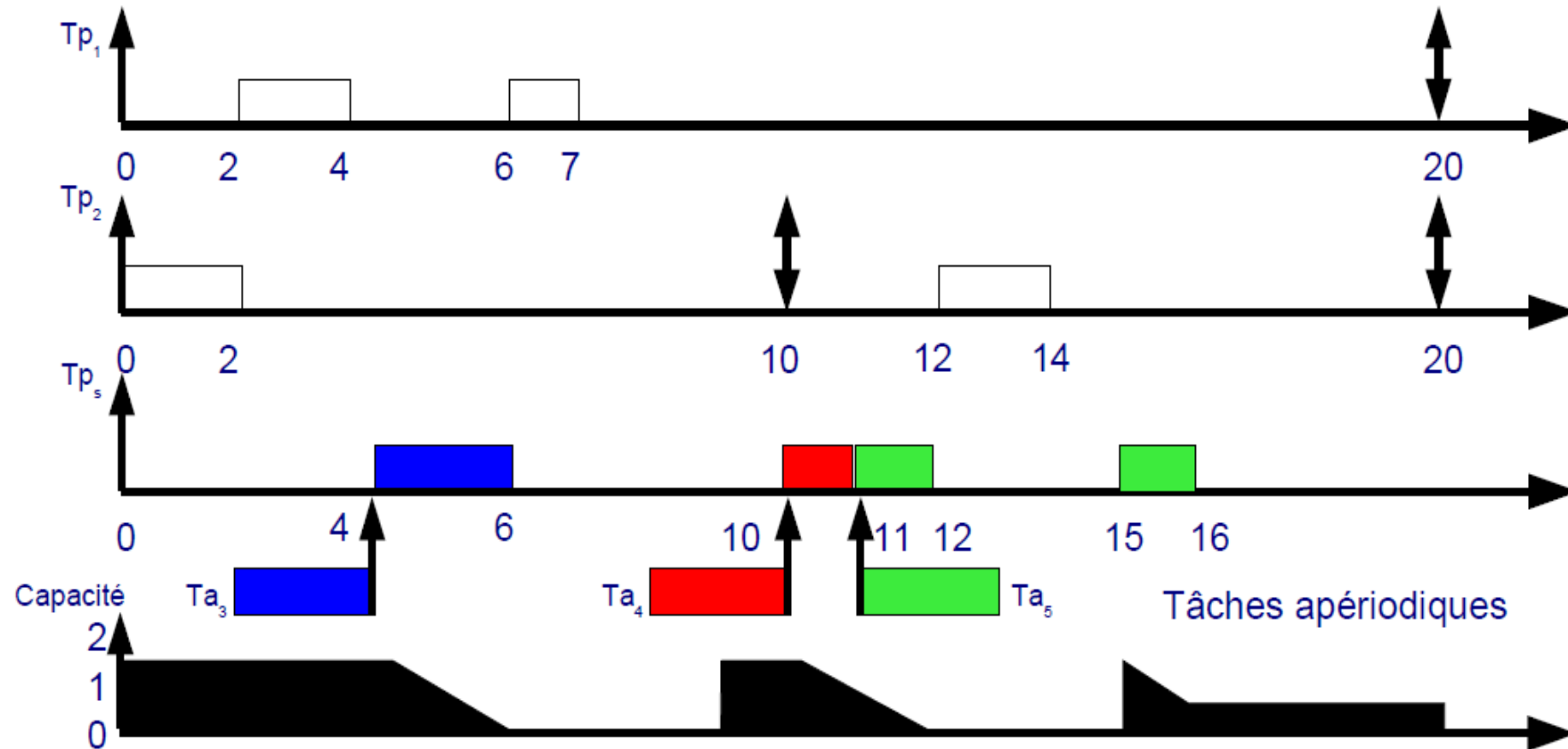
Tps ($r_0 = 0, C = 2, P = 5$)

➤ tâches apériodiques :

Ta_3 ($r = 4, C = 2$),

Ta_4 ($r = 10, C = 1$),

Ta_5 ($r = 11, C = 2$)



Tâches apériodiques à contraintes relatives

Voir ordonnancement des tâches apériodiques et à contraintes strictes

- ❑ Serveur différé
- ❑ ordonnancement conjoint